

Virtuelle Speicherverwaltung

Adressräume, Page Tables und Page Faults verstehen

3.05

Eine Adresse durch das System verfolgen

01 Adressräume einordnen

User Space und Kernel Space pro Prozess unterscheiden.

02 Übersetzung erklären

Translation Lookaside Buffer, Page Tables und Page Frames zusammendenken.

03 Fehlerpfade verstehen

Page Faults, Demand Paging und Copy-on-Write erklären.

Praxisziel

Einen realen Prozess mit `/proc/<pid>/maps` und `/proc/<pid>/smaps` untersuchen.

Virtueller Speicher trennt Sicht und Schutz

01 Eigene Sicht

Jeder Prozess arbeitet in einem scheinbar zusammenhängenden Adressraum.

02 Schutzgrenzen

Zugriffsrechte werden page-weise geprüft: lesen, schreiben, ausführen.

03 Flexible Platzierung

Virtuelle Pages können auf beliebige physische Page Frames zeigen.

04 Effiziente Nutzung

Speicher kann verzögert belegt, geteilt oder ausgelagert werden.

Jeder Prozess besitzt einen eigenen Adressraum

01 Prozess A

Stack, Heap, Code und Mappings bilden eine eigene virtuelle Sicht.

02 Prozess B

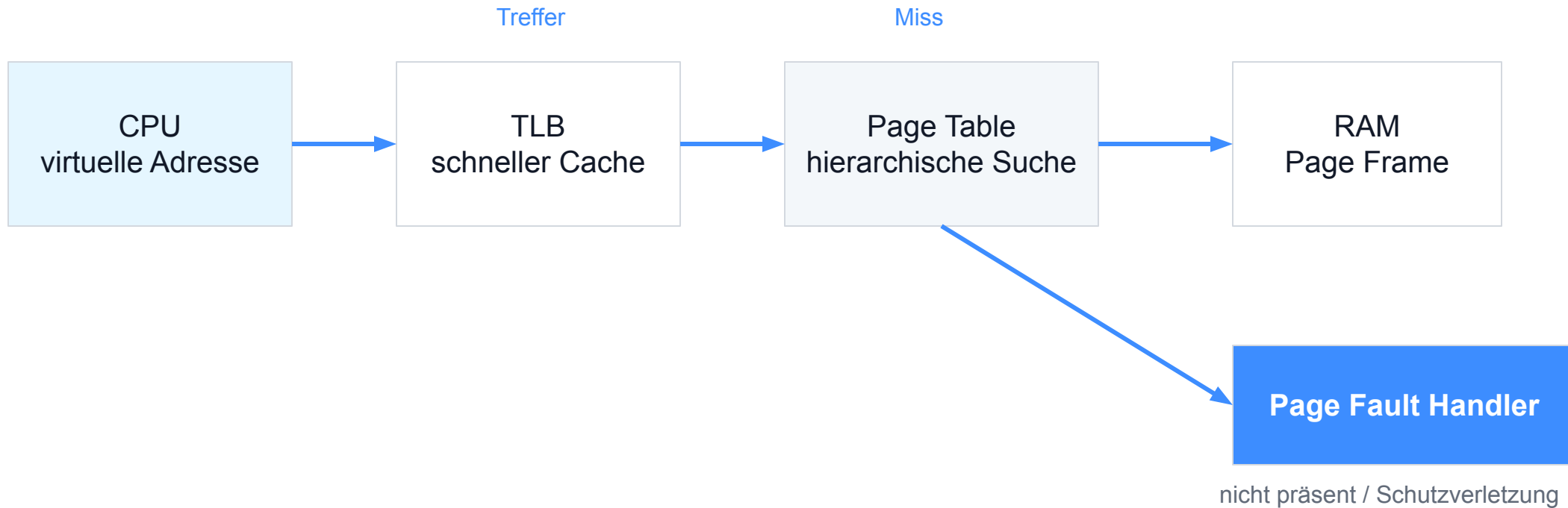
Gleiche virtuelle Adressen können auf andere Page Frames zeigen.

KERNEL SPACE

**Gemeinsam gemappt,
aber aus dem User Mode
nicht direkt zugreifbar**

Gleiche virtuelle Adresse bedeutet nicht gleicher physischer Speicher.

Die MMU übersetzt jede Speicherreferenz

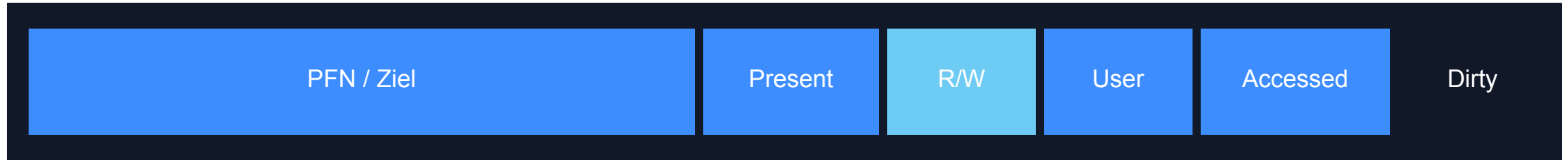


Linux modelliert Page Table als fünfstufige Hierarchie

01	PGD	oberste Ebene
02	P4D	architekturabhängig
03	PUD	grobe Bereiche
04	PMD	mittlere Ebene
05	PTE	Page Frame + Rechte

Nicht benötigte Ebenen können architekturenspezifisch zusammengefasst werden.

Ein Page-Table-Eintrag verbindet Ziel und Regeln



PFN	Welcher physische Page Frame ist gemeint?	Present	Ist die Übersetzung aktuell gültig?
R/W + User	Welche Zugriffe sind aus welchem Modus erlaubt?	Accessed + Dirty	Wurde gelesen oder geschrieben?

Page Faults sind nicht automatisch Fehler

- 01 CPU erkennt den Fault** Übersetzung fehlt oder der Zugriff verletzt ein Page-Recht.
- 02 Kernel prüft VMA und Rechte** Der Fault wird gegen den Prozessadressraum validiert.
- 03 Kernel löst auf oder verwirft** Demand Paging, Copy-on-Write und Stackwachstum sind legitime Fälle.
- 04 Ausführung geht weiter oder endet** PTE/TLB aktualisieren – oder bei ungültigem Zugriff ein Signal senden.

Der Page Fault ist ein Mechanismus; erst ein nicht auflösbarer Zugriff wird zum Prozessfehler.

Demand Paging und Copy-on-Write verschieben Arbeit

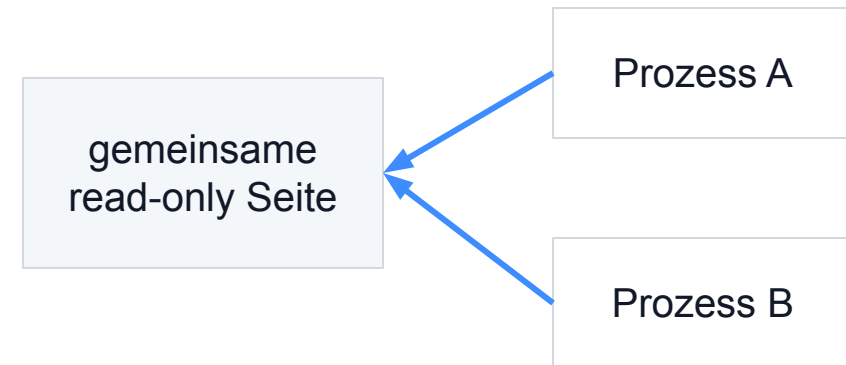
Demand Paging

Virtueller Bereich wird angelegt

Physische Seite folgt erst beim Zugriff

Dateiinhalt oder Nullseite wird bei Bedarf geladen

Copy-on-Write



Schreibzugriff → private Kopie

Beide Mechanismen tauschen sofortige Allokation gegen einen späteren Page Fault.

VMAs strukturieren den Prozessadressraum

01 mm_struct

Bündelt den gesamten virtuellen Adressraum eines Prozesses.

02 Virtual Memory Area (VMA)

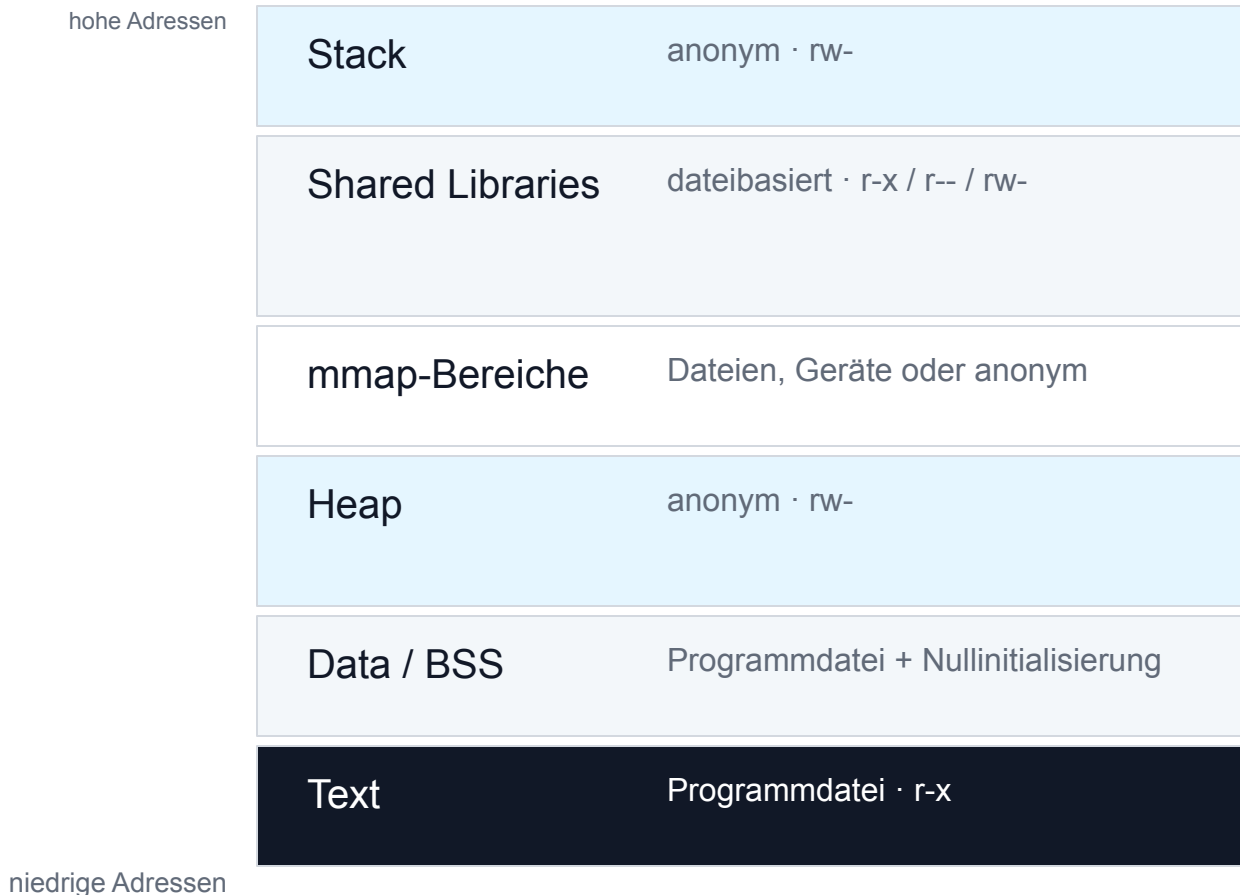
Beschreibt einen zusammenhängenden Bereich mit einheitlichen Regeln.

EINE VMA BESCHREIBT

A	Grenzen	Start- und Endadresse
B	Rechte	Lesen, Schreiben, Ausführen
C	Backing	anonym oder dateibasiert

Page-Table beschreiben einzelne Pages; VMAs beschreiben zusammenhängende Bereiche.

Ein Adressraum verbindet mehrere Speicherquellen



virtuell

- Nicht jeder Bereich ist vollständig resident.
- Nicht jeder Bereich ist privat.
- Nicht jeder Bereich stammt aus RAM.

/proc/<pid>/maps macht VMAs sichtbar

BEISPIELZEILE

```
7f2a9c000000-7f2a9c021000 rw-p 00000000 00:00 0 [heap]
```

01 Adressbereich

Start und Ende der VMA.

02 Rechte

r/w/x sowie privat oder geteilt.

03 Backing

Offset, Gerät/Inode und Pfad oder Name.

/proc/<pid>/smaps ergänzt die Bereiche um Residenz- und Verbrauchsdaten.

Page Größe bestimmt Reichweite und Verwaltungsaufwand

Kleine Page

weniger interner Verschnitt
mehr PTEs und TLB-Druck

Große Pages

weniger Übersetzungen
mehr Bindung pro Mapping

Transparent Huge Pages

Kernel kann geeignete Bereiche
automatisch mit großen Pages
abbilden.

Die passende Page Größe ist ein Trade-off – kein pauschales Optimum.

Praxis: Einen laufenden Prozess untersuchen

TERMINAL

```
$ PID=$$  
$ cat /proc/$PID/maps  
$ less /proc/$PID/smmaps
```

01 Prozess wählen

PID setzen und Existenz prüfen.

02 Bereiche lesen

Stack, Heap, Libraries und Text zuordnen.

03 Verbrauch prüfen

Virtuelle Größe und Residenz vergleichen.

Leitfrage: Welche Bereiche sind virtuell groß, aber nur teilweise resident?

Vier Aussagen tragen das Gesamtmodell

01 Virtuell ist eine Sicht

Ein Prozess arbeitet mit Adressen, nicht direkt mit RAM.

02 Übersetzung ist hierarchisch

TLB und Page Table verbinden virtuelle Pages mit Page Frames.

03 Page Faults sind Mechanismus

Der Kernel kann fehlende Übersetzungen legitim herstellen.

04 VMAs geben Struktur

mm_struct und VMAs beschreiben Bereiche, Rechte und Backing.

Nächster Schritt: Speicher im Kernel sicher allokieren und wieder freigeben.